数学与 CS 问答笔记

answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

November 13, 2026

Contents

1. 0101 测试 spec 建议：分区、子域与验收判据 4
   1.1. 费曼直觉 4
   1.2. 硬核机制 4
      1.2.1. 当前 strategy 的实际内容 4
      1.2.2. 分区的三条判据 4
      1.2.3. KV 的输入空间有哪些维度 5
      1.2.4. 建议的 strategy 骨架（结构，不是让你照抄） 5
      1.2.5. 当前测试覆盖对应到分区 6
      1.2.6. 第 50 行：现在必须修 6
      1.2.7. 测试组织：现在要不要拆表驱动 6
      1.2.8. 断言质量：现在能免费改的一件事 7
      1.2.9. 真实项目样本：`bytes.TestClone` 7
   1.3. 陷阱与易错 8
   1.4. 关键点 8
   1.5. 变式检验 8
   1.6. 同类归类 9
   1.7. Go 语言机制补充 9
   1.8. 本次操作与当前状态 9
2. test partition 检查：互斥、完整、非空 11
   2.1. 费曼直觉 11
   2.2. 硬核机制 11
      2.2.1. 当前分区 11
      2.2.2. 三项判据逐条判定 11
      2.2.3. 问题一：`nil` 与 `len == 0` 重叠 11
      2.2.4. 问题二：Get / Del 缺「key 合法性」维度 12
      2.2.5. 问题三：缺 history 分区 12
      2.2.6. 问题四：两个子域没有用例 13
      2.2.7. 问题五：第 48 行仍是假绿 13
      2.2.8. 问题六：拼写与空行 13
      2.2.9. 与 spec 的对应关系 13
      2.2.10. 建议的分区结构（骨架，文字自己写） 14
      2.2.11. 当前测试强度下的最小改法 14
   2.3. 陷阱与易错 14
   2.4. 关键点 15
   2.5. 变式检验 15
   2.6. 同类归类 15
   2.7. Go 语言机制补充 16
   2.8. 本次操作与当前状态 16
3. 为什么有了 null 的危害，Go 和 TypeScript 仍保留空值 17
   3.1. 费曼直觉 17
   3.2. 硬核机制 17

3.2.1. 先对齐术语 17
3.2.2. 课程到底反对什么 17
3.2.3. 为什么语言仍然需要「空」 18
3.2.4. Go 的路线：nil 是零值，不是万能 null 18
3.2.5. TypeScript 的路线：不新增 null，而是把可空性显式化 19
3.2.6. 为什么彻底消灭空值很难 20
3.2.7. 那课程的建议还有用吗 20
3.3. 陷阱与易错 20
3.4. 关键点 21
3.5. 变式检验 21
3.6. 同类归类 22
3.7. Go 语言机制补充 22
3.8. 本次操作与当前状态 22
4. 0101 test partition 二次检查：维度错配与未修项 24
4.1. 费曼直觉 24
4.2. 硬核机制 24
4.2.1. 当前分区 24
4.2.2. 维度 × 方法 适用性矩阵 24
4.2.3. 证据：Set 的行为不随「key 是否存在」变化 25
4.2.4. 仍未修的互斥问题 25
4.2.5. 空的子域 25
4.2.6. 仍缺的两块 26
4.2.7. 第 47 行仍是假绿 26
4.2.8. 拼写与空行 26
4.2.9. 修正后的分区结构（骨架） 27
4.2.10. 验收清单 27
4.3. 陷阱与易错 27
4.4. 关键点 28
4.5. 变式检验 28
4.6. 同类归类 28
4.7. Go 语言机制补充 29
4.8. 本次操作与当前状态 29
Index of Listings 30

1. 0101 测试 spec 建议：分区、子域与验收判据

1.1. 费曼直觉

先分清两件事：**测试代码**和**测试 spec（testing strategy）**不是一回事。

1. 测试代码回答「跑了什么断言」；
2. testing strategy 回答「你打算怎么切输入空间、切得全不全、哪些这次不切」。

后者像体检项目单：先列要查的器官（分区），再列每个器官查哪几项（子域），最后写明「本次不查」（阶段未清单）。没有这张单子，测试就会变成「想到哪写到哪」，覆盖率再高也不知道漏了什么。

> 📌 **核心要点** 当前 `kv_test.go` 的 strategy 有三个 `TODO`（Set、history、未覆盖子域），第 50 行还是假绿。所以现在的状态不是「测试不够多」，而是**测试计划还没写**。 先把 strategy 写完整，再决定加不加用例。

1.2. 硬核机制

1.2.1. 当前 strategy 的实际内容

```
partition on operation:
  Get: use key state
  Set: TODO（分叉点：键是否已存在 / 值是否变化）
  Del: use key state
partition on key state:
  key in mem
  key not in mem
partition on history（调用历史）:
  TODO
当前未覆盖的子域（阶段末再决定是否加测）:
  TODO
```

Listing 1: `kv_test.go` 第 6–20 行：两个分区有内容，两个是空 TODO

分区	状态	缺口
operation	部分	`Set` 只有 TODO；`Open` / `Close` 未列
key state	完整	`in mem` / `not in mem` 互斥且覆盖全部键
history	空 TODO	调用序列完全没有分区
未覆盖子域	空 TODO	没有说明「知道缺什么」

1.2.2. 分区的三条判据

判据	含义	反例
互斥	一个输入只落在一个子域	同时把「键存在」和「键不存在」放进同一个用例
完整	所有可能的输入都有归属	只测非空键，忘了空键（阶段末可以标注）

非空	每个子域至少有一个代表	写了 `Set: existing`，但没有任何用例

注意「完整」不等于「现在全测」。完整的 strategy 可以写「空键暂时不测」，但必须写明这是**决定**，不是遗漏。这就是阶段末清单的作用。

1.2.3. KV 的输入空间有哪些维度

维度	取值	当前是否测
operation	Get / Set / Del（Open / Close 阶段末）	是
key state	in mem / not in mem	是
value state	非空 / 空 / nil	否（阶段末）
history	Set→Get / Set→Del→Get / Del→Del / Set→Set→Get	部分
输入合法性	正常 / nil key / 空 key	否（阶段末）
别名	改入参 / 改返回值	否（阶段末）

不要做完整笛卡尔积：6 个维度相乘会产生几十个用例，多数没有意义。做法是**选有代表性的组合**，并在 strategy 里说明为什么选它们。

1.2.4. 建议的 strategy 骨架（结构，不是让你照抄）

```
/*
 * Testing strategy
 *
 * Partition the input space:
 *   by operation: Get / Set / Del
 *     Get: key state
 *     Set: key state (new / existing)      <- TODO(阶段末)
 *     Del: key state
 *   by key state: in mem / not in mem
 *   by history:
 *     Set -> Get
 *     Set -> Del -> Get
 *     Del -> Del
 *
 * Cover the subdomains:
 *   Get: key in mem (hit), key not in mem (miss)
 *   Set: new key
 *   Del: key in mem (hit), key not in mem (miss)
 *   history: Set -> Get, Set -> Del -> Get, Del -> Del
 *
 * Not covered yet (TODO(阶段末), after the author ships this module's
tests):
 *   nil key / empty key, nil value / empty value,
 *   same-key same-value, aliasing, Open / Close lifecycle, zero value
 */
```

Listing 2: strategy 的骨架：分区 → 子域 → 阶段末清单；文字由你自己写

三段的用途不同：

1. **Partition**：说清维度，不列用例；
2. **Cover the subdomains**：写清这次真正覆盖的组合；
3. **Not covered yet**：写清「决定不覆盖」的部分和触发条件。

1.2.5. 当前测试覆盖对应到分区

分区	子域	用例位置	断言
Get	key in mem	第 38–39 行	`val == v1 && ok && err == nil`
Get	key not in mem	第 41–42 行	`!ok && err == nil`
Set	new key	第 35–36 行	`updated && err == nil`
Del	key in mem	第 47–48 行	`deleted && err == nil`
Del	key not in mem	第 44–45 行	`!deleted && err == nil`
history	Set→Get	第 35、38 行	组合覆盖
history	Set→Del→Get	第 47、50 行	**查错键，假绿**
history	Del→Del	无	阶段末或现在补一条

`Del→Del` 其实是当前强度下就能测的：第二次 `deleted` 必须是 false。 它与「重复删除」的语义直接相关，不需要等未来测试。可以现在加一条。

1.2.6. 第 50 行：现在必须修

```
    updated, err = kv.Del(k1Key)
    assert.True(t, updated && err == nil)

    _, ok, err = kv.Get(xxxKey)
    assert.True(t, !ok && err == nil)
```

Listing 3: 删的是 `k1`，查的是 `xxx`：删除逻辑失效时这条断言仍然通过

修法只有一行：`Get(k1Key)`。 验证方式：临时注释掉 `Del` 里的 `delete(kv.mem, ...)`，测试必须变红；恢复后必须变绿。

> ⚠️ **陷阱预警 假绿比没测更危险。** 没测你知道「这里没保护」；假绿会让你以为「已经保护了」。 第 50 行就是后者。

1.2.7. 测试组织：现在要不要拆表驱动

Go 的惯例是表驱动 + 命名子测试（`t.Run`），但项目当前是单个冒烟函数。 按「当前测试强度」原则，**不强制现在重构**；如果要拆，建议：

做法	好处	代价
保持单个 `TestKVBasic`	改动最小，当前测试全绿	失败时只能看到「哪一行」
拆成 `TestKV_Get` / `TestKV_Set` / `TestKV_Del`	与方法名对齐，失败定位清楚	要重写现有断言

表驱动 + `t.Run`	每个子域一个命名用例，`-run` 可过滤	准备工作多，属于阶段末重构

建议顺序：先修假绿 → 再写 strategy → 阶段末再拆表驱动。 不要为了「看起来专业」先重构，那会把「策略」和「结构」两件事混在一起。

1.2.8. 断言质量：现在能免费改的一件事

`assert.True(t, a && b && c)` 失败时只输出 `Should be true`， 三个条件哪个错、实际值是什么都看不到。改成：

```
    require.NoError(t, err)
    assert.True(t, updated)
    assert.Equal(t, []byte{'v', '1'}, val)
```

Listing 4: 拆断言： `require` 用于后续断言依赖的前置， `assert` 用于独立的属性

`require` 失败会调用 `t.FailNow()`，适合 `Open` 这种「失败就没法继续」的调用； `assert` 失败会继续，适合收集多个独立问题。

> ⚠️ **陷阱预警 不要在父测试创建 `assert.New(t)` 再在子测试里复用。** testify 的实例绑定创建时的 `*testing.T`， 子测试里的失败会被归因到父测试，而子测试仍然显示 `PASS` 。 每个 `t.Run` 内部各自 `assert.New(t)` 。

1.2.9. 真实项目样本： `bytes.TestClone`

```
    var cloneTests = [][]byte{
      []byte(nil),
      []byte{},
      []byte(strings.Repeat("a", 42))[:0],
      []byte(strings.Repeat("a", 42))[:0:0],
      []byte("short"),
    }
    for _, input := range cloneTests {
      clone := Clone(input)
      if !Equal(clone, input) {
        t.Errorf("Clone(%q) = %q; want %q", input, clone, input)
      }
      if input == nil && clone != nil {
        t.Errorf("Clone(%#v) return value should be equal to nil slice.",
input)
      }
      if input != nil && clone == nil {
        t.Errorf("Clone(%#v) return value should not be equal to nil slice.",
input)
      }
    }
```

Listing 5: GOROOT `src/bytes/bytes_test.go` 的 `TestClone` （节选）：nil、空、零容量各自成组

它值得借的地方：

1. 把 `nil`、空切片、零容量切片分开列，不合并成「空」；
2. 每条断言对应一个语义（内容相等、nil 进 nil 出、非 nil 不出 nil）；
3. 还有一条 `unsafe.SliceData` 比较，验证「不共享底层数组」——这正是 `Safety` 的测试模板， 阶段末补别名测试时可以照这个思路。

1.3. 陷阱与易错

⚠️ **陷阱预警 覆盖率不是分区。** 100% 行覆盖也可以完全没测「键不存在时删除」。strategy 先写维度，再谈覆盖率。

⚠️ **陷阱预警 按实现写测试。** 先读 `Set` 的实现再写断言，会把「写后比较恒真」固化成规格。 测试应该来自 spec，不是来自代码。

⚠️ **陷阱预警 把「未覆盖」留成空 TODO。** 空 TODO 让人分不清「知道缺」还是「忘了写」。 写成 `TODO(阶段末)：缺 X、Y，等作者的测试给完再补` 。

⚠️ **陷阱预警 一个断言塞三个条件。** 失败时只显示 `Should be true`，排查成本极高。拆成 `require.NoError` + `assert.Equal` + `assert.True` 。

1.4. 关键点

📌 **核心要点** 0101 的 testing strategy 要做三件事： 把 `Set` 与 `history` 的分区写出来；把当前覆盖的子域列全； 把不覆盖的维度写成带触发条件的 `TODO(阶段末)` 。 测试代码先只修第 50 行的假绿，重构留给阶段末。

步骤	动作	验收
1	修第 50 行：`xxx` → `k1`	注释 `delete` 后测试变红
2	写 `Set` 与 `history` 的分区	两个空 TODO 消失
3	列当前覆盖的子域	每条对应到具体用例
4	写阶段末清单与触发条件	没有空 TODO
5	拆 `assert.True(a && b)`（可选）	失败信息能定位

1.5. 变式检验

变式 1：`Set` 的分区怎么写才互斥又完整？ 按 key state 分「新键 / 已存在」，再按 value 是否相同分（阶段末）。 两个维度组合出四个子域，但当前强度只覆盖「新键」，其余写进阶段末。

变式 2：`history` 分区最少要覆盖哪几条？ `Set→Get`（正常链路）、`Set→Del→Get`（删除后不可见）、`Del→Del`（幂等性/重复删除）。这三条覆盖了「状态在调用之间如何变化」的核心。

变式 3：为什么 `Del→Del` 现在就能测，而 `nil key` 要等？ 重复删除只依赖当前实现已有的行为（第二次返回 false），不需要新增接口或错误值；`nil key` 涉及「nil 算错误还是普通键」的决策，属于未来测试会定义的维度。

变式 4：如果作者以后要求 `Set` 返回 `updated=false` 当值未变，strategy 怎么变？ `Set` 的分区加上「值相同 / 值不同」，用例从 1 个变成 3 个；当前 strategy 里已经预留了 `TODO(阶段末)`，届时不改结构，只把 TODO 变成用例。

变式 5：表驱动一定要现在做吗？不必。表驱动是**组织方式**，不是**覆盖范围**。当前强度下先保证断言有效，拆表驱动可以在阶段末和新增用例一起做。

1.6. 同类归类

测试类型	解决什么	本项目何时用
冒烟测试	主链路能跑通	当前 `TestKVBasic`
表驱动 + 子测试	多个子域、失败可定位	阶段末重构
边界测试	空、nil、重复、缺失	阶段末（nil / 空值 / 重复删除）
属性测试	对随机输入验证不变量	暂不需要
变异测试	验证测试真的能红	修第 50 行时手动做一次

1.7. Go 语言机制补充

同包白盒测试。 `kv_test.go` 声明 `package db`，可以直接构造 `KV{}`、调用未导出字段。`go test` 只编译同包测试文件，不影响生产构建。

命名子测试与 `-run`。 `t.Run("hit", ...)` 可以用 `go test -run 'TestKV_Get/hit'` 单独跑，失败时输出会带子测试名。

`assert` 与 `require`。 `assert` 失败继续执行，`require` 失败调用 `t.FailNow()`。前置条件（如 `Open` 成功）用 `require`，独立属性用 `assert`。

`-count=1` 禁用测试缓存。改了实现但没改测试时，`go test` 可能复用缓存结果；验证修复一定要带 `-count=1`。

测试文件命名。 `kv.go` 对应 `kv_test.go`（按源文件命名，不按函数命名）。同包测试可以访问未导出字段，适合当前白盒验证。

6.031 对照。本题对应 R04 测试（分区、黑盒/白盒、回归测试）与 R11 的「RI 要有检查」；Go 的 `testing` 包不提供分区概念，分区只存在于 strategy 注释与用例组织里。

1.8. 本次操作与当前状态

已做的文件操作：

1. 加载 `golang-testing` skill，读取表驱动、命名子测试、assert/require、 测试文件命名等硬规则；
2. 读取 `internal/db/kv_test.go` 当前 strategy 与 7 条断言；
3. 读取 GOROOT `src/bytes/bytes_test.go` 的 `TestClone` 作为拷贝语义测试样本；
4. 新建本文件 `note/2026-11-13.typ`（当日首个问题），沿用完整头部；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. 本节为测试策略建议，未新增命令验证；
2. 引用的 `TestClone` 原文来自 `/usr/lib/go/src/bytes/bytes_test.go` 第 2500–2527 行；
3. `golang-testing` skill 的硬规则：表驱动必须用命名子测试（`name` + `t.Run`）；不得在父测试创建 `assert.New(t)` 后跨子测试复用；
4. `typst compile 2026-11-13.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 现在做：第 50 行 `xxx` → `k1`，并用「注释掉 `delete` 看红」验证；
2. 现在做：把 `Set` 与 `history` 两个空 TODO 写成分区；
3. 现在做：把「未覆盖子域」写成带触发条件的阶段末清单；
4. 可选：拆 `assert.True(a && b && c)`，让失败信息可定位；
5. 阶段末：拆表驱动 + 子测试、补 nil/空值/别名用例。

2. test partition 检查：互斥、完整、非空

2.1. 费曼直觉

先给判定：**方向对了，但有一处重叠、两处缺失、两个子域没有用例。**

你这版从「按 operation 列」改成「按方法和 key 状态列」，更接近 6.031 的分区写法。但分区有三条硬判据：

1. **互斥**：一个输入只能落在一个子域；
2. **完整**：所有输入都有归属；
3. **非空**：每个子域至少有一个代表用例，或者明确标为「暂不覆盖」。

现在的分区在「互斥」上有一处硬伤，在「非空」上有两个空子域。

> 📌 **核心要点** `key is nil` 和 `key length is 0` 不是互斥的：nil 切片的长度就是 0。一个 nil key 同时满足两个子域，分区就不再是分区，而是「两组描述」。

2.2. 硬核机制

2.2.1. 当前分区

```
1   Get(), Del():
2     partition on whether key in store:
3       - key exist in store.
4       - key does not exist in store.
5
6   Set():
7     partition on key value:
8       - key is nil
9       - key length is 0
10      - key length is more than 0
```

Listing 6: `kv_test.go` 第 8–17 行：两个分区，一个重叠

2.2.2. 三项判据逐条判定

分区	判定	问题
Get / Del 的 in / not in	互斥、完整	只描述 store 状态，不覆盖 nil key
Set 的三类	**不互斥**	`nil` 的 `len` 也是 0，与前两类重叠
Set 的三类	完整	覆盖所有 `[]byte`（修正互斥后）
Set 的三类	**非空**	`nil` 与 `len == 0` 没有任何用例
history	**缺失**	没有调用序列分区
未覆盖子域	**缺失**	没有阶段末清单

2.2.3. 问题一：`nil` 与 `len == 0` 重叠

`len(nil)` 在 Go 里就是 0。所以一个 nil key 会同时落进两个子域：

```
var k []byte          // k == nil, len(k) == 0
[]byte{}              // k != nil, len(k) == 0
[]byte{'k'}           // k != nil, len(k) == 1
```

Listing 7: 三种输入：前两种长度都是 0，但 nil 性不同

互斥的分法只有一种：

```
partition on key:
  - key is nil
  - key is non-nil and len(key) == 0
  - key is non-nil and len(key) > 0
```

Listing 8: 互斥且完整的三分法：先判 nil，再判长度

写分区时可以用一个判别顺序来检查互斥：**每个子域的谓词两两不能同时为真。** `key == nil` 与 `len(key) == 0` 同时为真，所以它们不能并列。

2.2.4. 问题二：Get / Del 缺「key 合法性」维度

Get 的 spec 有三个分支（存在 / 不存在 / nil），Del 的 spec 也有三个（存在 / 不存在 / nil）。但分区只写了「是否在 store」，把 nil 藏进了「不存在」里。

这会造成两个后果：

1. `Del(nil)` 返回 `ErrKeyIsNil`，而 `Del(不存在的键)` 返回 `false, nil` —— 两者都落在「not in store」，但行为不同，分区无法区分；
2. `Get(nil)` 的 spec 说返回错误，实现返回 `nil, false, nil` —— 分区没有覆盖这个分支，所以测试也不会暴露它。

补法有两种：

做法	写法	代价
加正交维度	`partition on key validity: nil / non-nil`	分区变两个维度，用例增加
把 nil 归入 not in store 并说明	`key not in store (including nil)`	必须同时说明 nil 的错误行为

无论选哪种，**nil 的行为必须在 strategy 里出现一次**，否则它永远是「碰巧被测到」。

2.2.5. 问题三：缺 history 分区

上、上一版有 `partition on history`，这一版整段没了。调用序列是 KV 的核心维度：

```
partition on history:
  - Set -> Get
  - Set -> Del -> Get
  - Del -> Del
```

Listing 9: 最小 history 分区：正常链路、删除后不可见、重复删除

现在第 48 行查 `xxx`，恰好说明 history 分区没有写清：`Set -> Del -> Get` 的 Get 应该查被删的键，而不是另一个从未存在的键。

2.2.6. 问题四：两个子域没有用例

strategy 列了 Set 的 `nil` 与 `len == 0`，但测试里只有：

```
    updated, err := kv.Set([]byte{'k', '1'}, []byte{'v', '1'})
```

Listing 10: 当前唯一的 Set 调用：只覆盖「非空键」子域

按「非空」判据，这两个子域要么补用例，要么标 `TODO(阶段末)`。它们不是「未来接口」——`Set` 已经实现了这两个分支，spec 也写了，所以现在补两条断言成本很低；如果按你的测试强度原则推迟，就明确写：

```
    TODO(阶段末): Set 的 nil / empty key 子域暂不覆盖，等作者给完本模块测试。
```

Listing 11: 推迟也要留下可追溯的标记

2.2.7. 问题五：第 48 行仍是假绿

```
    updated, err = kv.Del([]byte{'k', '1'})
    assert.True(t, updated && err == nil)

    _, ok, err = kv.Get([]byte{'x', 'x', 'x'})
    assert.True(t, !ok && err == nil)
```

Listing 12: 删 `k1` 却查 `xxx`：删除逻辑失效时这条断言仍然通过

这和分区无关，但它是当前测试里唯一「测试本身写错」的地方，优先级高于补分区。

2.2.8. 问题六：拼写与空行

块注释开头有两个空的 `*` 行；`exisit` 出现两次。`gofmt -l` 目前无输出（它不检查拼写），但 `go doc` /评审会原样看到。

2.2.9. 与 spec 的对应关系

方法	SPEC 分支	STRATEGY 是否覆盖
`Get`	存在 / 不存在 / nil	前两个有，nil 无
`Set`	nil / 空键 / 非空键	列了三个，只测了非空
`Del`	存在 / 不存在 / nil	前两个有，nil 无
`Open` / `Close`	阶段末	未列

strategy 的分区应该**至少**覆盖 spec 的每个分支一次。现在是「spec 三分支， strategy 两分支」，测试自然缺一角。

2.2.10. 建议的分区结构（骨架，文字自己写）

```
Get(), Del():
  partition on key validity:
    - key is nil
    - key is non-nil
  partition on whether key in store:
    - key exists in store
    - key does not exist in store

Set():
  partition on key:
    - key is nil
    - key is non-nil and len(key) == 0
    - key is non-nil and len(key) > 0

history:
  - Set -> Get
  - Set -> Del -> Get
  - Del -> Del

TODO(阶段末):
  - Set same key same value / different value
  - value is nil / empty
  - aliasing (input / return value)
  - Open / Close lifecycle, zero value
```

Listing 13: 修正后的骨架：互斥的三分法 + key 合法性 + history + 阶段末清单

2.2.11. 当前测试强度下的最小改法

顺序	动作	验收
1	第 48 行 `xxx` → `k1`	注释 `delete` 后测试变红
2	修 Set 分区为「nil / 非 nil 且空 / 非 nil 且非空」	两两谓词不同时为真
3	加 key validity 分区（或说明 nil 的归属）	spec 的每个分支都有归属
4	补 history 三条	第 48 行属于 Set→Del→Get
5	nil / 空键子域：补断言或标 `TODO(阶段末)`	没有空子域
6	修 `exisit` 与开头空 `*` 行	注释干净

第 2、3 步只改注释；第 5 步要么加两行断言，要么标 TODO。

2.3. 陷阱与易错

⚠️ **陷阱预警 分区不互斥，等于没分区。** 一个输入落在两个子域里，你就不知道失败时该归因给谁，也不知道「覆盖了」到底覆盖了什么。

⚠️ **陷阱预警 列了子域却没有用例。** strategy 不是愿望清单；每个子域要么有用例，要么有 `TODO(阶段末)`，不能沉默。

⚠️ **陷阱预警 只按 store 状态分区，漏掉输入合法性。** `Del(nil)` 和 `Del(不存在的键)` 都在「not in store」，但一个返回错误、一个返回 `false, nil`。

⚠️ **陷阱预警 strategy 不覆盖 spec 的分支。** spec 写三个分支，测试只分两类， 第三个分支就永远没有验收。

2.4. 关键点

📌 **核心要点** 当前分区有一处重叠（nil 与 len 0）、两处缺失（key validity、history）、两个空子域（Set 的 nil / 空键）。先修互斥，再补缺失，最后处理空子域。

判据	当前	目标
互斥	Set 的三类重叠	nil / 非 nil 空 / 非 nil 非空
完整	缺 nil 与 history	spec 每个分支都有归属
非空	Set 两类无用例	有用例或 `TODO(阶段末)`

2.5. 变式检验

变式 1：为什么不能把 `nil` 归入「len == 0」子域？可以归，但要写成「len == 0（含 nil）」，并说明 nil 会返回 `ErrKeyIsNil`、 空切片返回 `false, nil`——两者行为不同，合并后无法区分。 分区可以合并，**行为差异不能合并**。

变式 2：Get 的 nil 分支要不要单独一个子域？取决于 spec：如果 nil 返回错误，就必须单独列；如果 nil 等同空串、返回未命中， 可以归入「not in store」，但要在注释里说明「nil 与空串等价」。

变式 3：history 分区要列多少条才够？覆盖「状态在调用之间如何变化」的核心路径即可：Set→Get、Set→Del→Get、Del→Del。 更多序列（Set→Set→Get）属于阶段末。

变式 4：分区正确了，测试就一定有效吗？ 不一定。分区保证「切得全」，断言保证「测得到」。还要做一次变异检查： 注释掉 `delete`，第 48 行必须变红。

变式 5：如果现在不想补 Set 的 nil / 空键断言，怎么写才不算遗漏？ 写 `TODO(阶段末): Set 的 nil / empty key 子域暂不覆盖，等作者给完本模块测试`， 并在分区里保留这两个子域。留 TODO 是决定，不写才是遗漏。

2.6. 同类归类

分区维度	回答什么	本项目例子
状态	输入对象处于什么状态	key in mem / not in mem

合法性	输入是否被接受	nil / 非 nil 空 / 非 nil 非空
历史	之前发生过什么调用	Set→Get / Set→Del→Get
值	值的性质	空 / nil / 非空（阶段末）
别名	输入输出是否共享内存	改入参 / 改返回值（阶段末）

2.7. Go 语言机制补充

`len(nil slice) == 0`。nil 切片是合法的零长度切片，所以「按长度分区」天然会把 nil 和空切片合并。要区分 nil 性，必须先判 `key == nil`，再判 `len(key) == 0`。

`string(nil)` 与 `string([]byte{})` 都是空串。 这是「nil key 与空 key 在 map 里是同一个键」的根源。分区可以分开描述，但实现里它们最终落到同一个键。

`gofmt` 不检查注释拼写。 `exisit`、空 `*` 行都会原样保留；`gofmt -l` 无输出只代表格式合法，不代表内容正确。

测试文件命名。 `kv.go` 对应 `kv_test.go`；同包白盒测试可以直接构造 `KV{}`。当前测试函数只有一个 `TestKVBasic`，阶段末拆成 `TestKV_Get` 等更利于定位。

6.031 对照。 本题对应 R04 测试分区：互斥、完整、非空；R06 规格的每个分支都应在测试里有对应子域。分区的写法是注释，执行靠用例。

2.8. 本次操作与当前状态

已做的文件操作:

1. 读取 `internal/db/kv_test.go` 当前 strategy（第 3–19 行）与 7 条断言；
2. 运行 `gofmt -l internal/db/` 与 `go test ./internal/db/ -count=1` 确认基线；
3. 追加本一级标题到 `note/2026-11-13.typ`；
4. 未修改 `internal/db` 下任何代码或测试文件。

验证结果:

1. `gofmt -l internal/db/` ⟶ 无输出；
2. `go test ./internal/db/ -count=1` ⟶ `ok`；
3. 静态检查确认：Set 的 `nil` / `len == 0` 两个子域没有用例；
4. 静态检查确认：第 48 行仍查 `xxx`；
5. `typst compile 2026-11-13.typ` ⟶ 待编译确认（见下）。

需要你自己动手的:

1. 第 48 行 `xxx` ⟶ `k1`，并用「注释 `delete` 看红」验证；
2. Set 分区改成「nil / 非 nil 且空 / 非 nil 且非空」；
3. 补 key validity 分区，或说明 nil 归入 not in store；
4. 补 history 三条；
5. Set 的 nil / 空键子域：补断言或标 `TODO(阶段末)`；
6. 修 `exisit` 与开头空 `*` 行。

3. 为什么有了 null 的危害，Go 和 TypeScript 仍保留空值

3.1. 费曼直觉

先校准一个前提：**Go 没有 `null`，TypeScript 也没有新增 `null`**。

1. Go 的关键字表里没有 `null`；它有的是 `nil`，作为指针、接口、切片、map、channel、函数的**零值**。
2. TypeScript 是 JavaScript 的超集，`null` 和 `undefined` 是 JS 本来就有的；TS 做的是**给它们加类型约束**，而不是「加入」它们。

所以问题应该改写成：**课程批评空值的危害，为什么这些语言还保留「某种空值」？**

一句话回答：课程反对的不是「表示缺失的机制」，而是**隐式可空 + 用空值表示错误 + 空值当哨兵**。 语言保留机制，同时用类型系统、返回值约定和文档来约束它——约束得有好有坏。

> 📌 **核心要点** `null` 像一把能开所有门的万能钥匙：方便，但正因为它对所有类型都有效， 你无法从类型上知道「这扇门后面有没有东西」。 课程主张的是给钥匙加一把锁（`Option` / `ok` / `strictNullChecks`），而不是废掉钥匙。

3.2. 硬核机制

3.2.1. 先对齐术语

名称	语言	含义	类型系统里的位置
`null`	JavaScript	显式空值	一个原始值，可赋给任何类型（非 strict）
`undefined`	JavaScript	未定义 / 缺失	同上，且是未初始化变量的默认值
`strictNullChecks`	TypeScript	把 null / undefined 变成独立类型	需要显式 `T | null`
`nil`	Go	指针 / 接口 / 切片 / map / channel / func 的零值	不是关键字，是预声明标识符
`None`	Rust	`Option<T>` 的一个变体	类型的一部分，编译器强制处理
`Optional`	Java	容器类型，表示「可能没有」	不是语言级可空性

这张表最重要的两行是 Go 和 TypeScript：它们**没有**把「所有类型都可空」写进语言。

3.2.2. 课程到底反对什么

6.031 / 6.102 对 null 的批评可以拆成三层，严重程度递增：

问题	后果	课程建议
隐式可空	每个引用类型都可能是 null，类型不提供保护	用类型显式表达可空性
用 null 表示错误 / 缺失	`Map.get` 返回 null，无法区分「没有」和「值是 null」	用 `ok` / `error` / `Option`
null 当哨兵	同一个值承担两种语义	用独立类型或和类型

注意第一层和第三层的区别：Java 的 `Map.get` 同时踩了前两层（返回 null、无法区分缺失与 null 值）；Go 的 `(v, ok)` 路线避开了第二层，但 `nil` 仍然是一个合法值，第三层仍可能踩。

3.2.3. 为什么语言仍然需要「空」

原因	说明	例子
内存现实	指针的零地址、未初始化的 map / channel	`var m map[string]int` 是 nil
缺失是正常语义	查找未命中、可选字段、没有请求体	`Request.Body` 为 nil 表示无 body
零值便利	`var x T` 可以直接用，不需要构造函数	`sync.Mutex` 零值可用
替代方案有成本	`Option` 需要标签位 / 装箱，改变 ABI 与 API	Rust 的 `Option<&T>` 零成本，但 `Option<T>` 不一定
兼容与演化	Go 1 兼容承诺；TS 不能破坏 JS 生态	不能把 `null` 从 JS 删除

五条里最硬的是最后两条：**语言不是从零设计的。** Go 要兼容 C 风格的指针与零值，TypeScript 要兼容整个 JavaScript 生态。

3.2.4. Go 的路线：nil 是零值，不是万能 null

Go 的 `nil` 有三个限制，正好对应课程批评的三层：

1. **只对特定类型合法**：不能把 `nil` 赋给 `int`、`string`、struct；
2. **零值语义**：`var p *T` 是 nil、`var m map[K]V` 是 nil、`var c chan T` 是 nil；
3. **用返回值表达缺失**：`(value, ok)` 或 `error`，而不是「返回 nil 表示没有」。

真实样本一：`sync.Map.Load` 用 `ok` 而不是「返回 nil 表示未命中」。

```
// Load returns the value stored in the map for a key, or nil if no
// value is present.
// The ok result indicates whether value was found in the map.
func (m *Map) Load(key any) (value any, ok bool)
```

Listing 14: GOROOT `src/sync/map.go`：未命中用 `ok` 表达，不产生错误

真实样本二：`Request.Body` 用 nil 表示「没有请求体」，但只在 client 侧。

```
    // For client requests, a nil body means the request has no
    // body, such as a GET request. The HTTP Client's Transport
    // is responsible for calling the Close method.
    //
    // For server requests, the Request Body is always non-nil
    // but will return EOF immediately when no body is present.
    // The Server will close the request body. The ServeHTTP
    // Handler does not need to.
    Body io.ReadCloser
```

Listing 15: GOROOT `src/net/http/request.go`：nil 的含义按角色（client / server）明确规定

第二条样本很关键：同样是「没有 body」，client 用 nil 表示，server 用「非 nil 但立即 EOF」表示。语言允许 nil，但**文档必须规定它在每个角色下的含义**——这正是课程要求的「别让空值语义含糊」。

真实样本三：`context.Done()` 返回 nil 表示「永远不会被取消」。

```
    // Done returns a channel that's closed when work done on behalf of this
    // context should be canceled. Done may return nil if this context can
    // never be canceled.
    Done() <-chan struct{}
```

Listing 16: GOROOT `src/context/context.go`： nil 表示「没有这个能力」

3.2.5. TypeScript 的路线：不新增 null，而是把可空性显式化

TypeScript 官方对 `strictNullChecks` 的说明只有两句，但把问题说透了：

```
    When strictNullChecks is false, null and undefined are effectively
    ignored by the language. This can lead to unexpected errors at runtime.

    When strictNullChecks is true, null and undefined have their own
    distinct types and you will get a type error if you try to use them
    where a concrete value is expected.
```

Listing 17: TypeScript 官方文档：`strictNullChecks` 的两种世界

同一个 `Array.find`，在两种模式下签名不同：

```
    // strictNullChecks: true
    find(predicate): S | undefined

    // strictNullChecks: false
    find(predicate): S
```

Listing 18: 官方文档的简化示例：关闭检查时，`undefined` 从类型系统里消失

所以 TypeScript 不是「加入 null」，而是**承认 JS 已经有 null / undefined，然后让它们在类型层面显式化**。代价是：

1. `strictNullChecks` 不是默认（新项目通常通过 `strict` 打开，老项目可能没有）；
2. `any` 会吞掉可空性；

3. 非空断言 `!` 可以绕过检查；
4. `null` 与 `undefined` 双轨，仍然容易混。

3.2.6. 为什么彻底消灭空值很难

成本	说明	例子
表示成本	`Option<T>` 可能需要额外的标签位或装箱	Rust 对引用做了 niche 优化，对整数没有
ABI / 互操作	C、JS、Java 生态都用空值	cgo、Web API、JDBC
兼容性	不能破坏现有代码	Go 1 承诺；TS 是 JS 超集
人体工学	每处都写 `Option` 很啰嗦	大量可选字段的配置结构
零值文化	Go 鼓励零值可用	`sync.Mutex` 、 `bytes.Buffer`

这不是「语言设计者不知道 null 的危害」，而是**在兼容、性能、人体工学之间做的取舍**。Rust 能彻底去掉 null，是因为它没有历史包袱，并且用 `Option` + 模式匹配把成本转给了类型系统。

3.2.7. 那课程的建议还有用吗

有，而且正好对应 Go / TS 的约束手段：

课程建议	Go 的做法	TypeScript 的做法
不要用 null 表示正常控制流	`(v, ok)` / `error`	`strictNullChecks` + 收窄
让可空性显式	`nil` 只对特定类型合法；文档写明含义	`T | null` / `T | undefined`
不要用空值当哨兵	`ErrKeyIsNil` 而不是「返回 nil 表示错误」	用联合类型区分状态

问题从来不在 nil 本身，而在「同一个值承担两种语义」。 `internal/db` 当前正好踩了这一条：

```
Set(k, nil)   // nil 是合法值
Get(k)        // 未命中时 val == nil，命中 nil 值也是 val == nil
```

Listing 19: nil 同时表示「值是 nil」和「键不存在」——这正是课程批评的第三层

`Get` 用 `ok` 区分了这两种情况，所以合同仍然可判；但 spec 里如果只写「未命中时 `val == nil` 」，读者就会误以为 `val == nil` 等价于未命中。 这正是为什么 `ok` 必须被写成「唯一命中判据」。

3.3. 陷阱与易错

⚠️ **陷阱预警 把 Go 的 nil 当 JavaScript 的 null。** nil 只对指针、接口、slice、map、channel、func 合法；`var x int = nil` 直接编译错误。类型系统已经排除了一整类 null 问题。

⚠️ **陷阱预警 typed nil 接口陷阱。** `var p *T = nil; var i I = p` 之后 `i != nil` 为真，但调用方法会 panic。nil 接口与「接口里装着 nil 指针」是两件事。

⚠️ **陷阱预警 nil map 可读不可写。** 读 nil map 返回零值，`delete` 是 no-op，写入 panic。`KV{}` 的 `mem` 是 nil，所以 `Set` panic、`Get` / `Del` 不 panic。

⚠️ **陷阱预警 `strictNullChecks` 不是默认开启。** 老项目可能关着；关着时 `null` / `undefined` 被类型系统忽略（官方原文），运行时错误照旧。

⚠️ **陷阱预警 非空断言 `!` 是绕过检查，不是证明。** `user!.name` 只是让编译器闭嘴，运行时该崩还是崩；它相当于把 TS 退回非 strict 模式。

3.4. 关键点

📌 **核心要点** Go 没有 null，只有 nil（零值）；TypeScript 没有新增 null，只是给 JS 的 null / undefined 加类型约束。课程反对的是「隐式可空 + 用空值表示错误 + 空值当哨兵」，不是反对「表示缺失」本身。语言保留机制，约束手段各有强弱。

语言	空值机制	约束手段
Go	`nil`（零值）	`(v, ok)`、`error`、类型限制、文档
TypeScript	`null` / `undefined`（继承自 JS）	`strictNullChecks`、`?.`、`??`、收窄
Java	`null`（隐式可空）	`Optional`、注解、静态分析
Rust	`Option<T>`（无 null）	模式匹配、编译器强制

3.5. 变式检验

变式 1：Go 为什么不把 nil 从语言里去掉？ `nil` 是指针、map、channel 等类型的零值；去掉它就要给这些类型加 `Option`，改变 ABI、零值语义和与 C 的互操作。Go 选择保留 nil，用返回值和文档约束。

变式 2：TypeScript 的 `strictNullChecks` 算不算「解决」了 null 问题？ 算「把隐式可空变成显式可空」，不算「消灭 null」。它能在编译期抓住大部分未处理的可空值，但 `any`、`!` 和运行时来源仍能绕过。

变式 3：为什么 `Map.get` 返回 null 在 6.031 里是反面教材？ 因为它无法区分「键不存在」和「键存在但值是 null」。Go 的 `sync.Map.Load` 返回 `(value, ok)`，用 `ok` 区分，正好是正面样本。

变式 4：Rust 没有 null，为什么还需要 `Option`？ `Option<T>` 把「可能没有」编码进类型，编译器强制处理；它不是 null，因为不能直接当 `T` 用，必须先 `match` 或 `?`。

变式 5：本项目的 `Get` 用 `ok` 算不算遵守了课程建议？算。它没有用 `nil` 表示未命中，而是用 `ok` 表达存在性。 但 spec 必须写明「`val == nil` 不等于未命中」，否则 `nil` 作为值的语义仍会混淆。

3.6. 同类归类

方案	可空性	代价
Java `null`	隐式可空	NPE 靠运行时暴露
JS `null` / `undefined`	隐式可空，双轨	运行时 TypeError
TS `strictNullChecks`	显式可空	需要开启，`any` / `!` 可绕过
Go `nil`	显式类型限制 + 零值	typed nil、nil map 写 panic
Rust `Option<T>`	类型级可空	需要显式处理，API 更冗长

3.7. Go 语言机制补充

零值。 Go 规范规定：布尔零值是 `false`，数值零值是 `0`，字符串零值是空串， 指针、函数、接口、切片、channel、map 的零值是 `nil` 。 所以 `var x T` 永远可用（至少语法上），不需要构造函数。

`nil` 可赋值的类型。只有指针、接口、slice、map、channel、func。 `nil` 不能赋给 `int` 、`string` 、 struct、 array。

typed nil。 接口值由「动态类型 + 动态值」组成； `var p *T = nil; var i I = p` 之后， `i` 的动态类型是 `*T` ，动态值是 nil，所以 `i == nil` 为 false，调用方法会 panic。判断要用类型断言或反射。

`(v, ok)` 与 `(v, error)` 的分工。 查找未命中用 `ok` （正常结果）， 失败用 `error` （异常结果）。 `sync.Map.Load` 与 `json.Unmarshal` 分别是两种样板。

`errors.Is(err, nil)`？不要这样写 ;直接 `err == nil` 。如果是 typed nil 错误值，`err != nil` 也可能成立——这是同一个接口陷阱的另一面。

6.031 对照。 本题对应 R06 规格（异常与 null 属于规格的一部分）、R07 设计规格（用类型表达可空性）。课程批评的是 Java 式隐式可空； Go 与 TS 的路线不同：一个用零值 + 返回值，一个用类型系统收窄。

3.8. 本次操作与当前状态

已做的文件操作：

1. 用 `go doc -src` 读取 `sync.Map.Load` 、 `net/http.Request.Body` 、 `context.Context.Done` 的原文注释；
2. 抓取并阅读 TypeScript 官方 `strictNullChecks` 文档；
3. 追加本一级标题到 `note/2026-11-13.typ` ；
4. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `go doc -src net/http.Request` → 确认 client 侧 nil body 表示无 body、 server 侧 Body 永远非 nil 且立即 EOF；
2. `go doc context.Context` → 确认 `Done` 可能返回 nil，表示永不被取消；
3. TypeScript 官方文档 → 确认 `strictNullChecks` 关闭时 null / undefined 被语言忽略，开启时成为独立类型；
4. `typst compile 2026-11-13.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 在 `Get` 的 spec 里补一句「`ok` 是唯一命中判据， `val == nil` 也可能表示命中」；
2. 在 `Set` 的 spec 里写明 nil 是合法值；
3. 如果你希望项目不把 nil 当合法值，就在 `Set` 里拒绝空值，并在 RI 里加回值约束。

4. 0101 test partition 二次检查：维度错配与未修项

4.1. 费曼直觉

先给判定：**有一处真正的改进，也有一处新的维度错配，旧问题大多还在。**

改进是把 `Set` 也纳入了「key value」分区—— `Set` 确实按 nil / 空 / 非空分叉。 新问题是把 `Set` 也塞进了「key 是否在 store」分区，而 `Set` 根本不看这个状态。

打个比方：分区表像一排分类垃圾桶。「key 是否在 store」这只桶上写着「已存在 / 不存在」， `Get` 和 `Del` 会按标签把垃圾分开投； `Set` 却根本不看标签，抓起来就扔。把 `Set` 列进这只桶，不会让它变得更守规矩，只会让读表的人以为它会分叉。

> 📌 **核心要点** 分区的判据不是「这个维度听起来相关」，而是**改变这个维度，方法的输出会不会变**。 不会变，就不该列进该方法的分区。

4.2. 硬核机制

4.2.1. 当前分区

```
1   Get(), Del(), Set():
2     partition on whether key in store:
3       - key exist in store.
4       - key does not exist in store.
5
6     partition on key value:
7       - key is nil
8       - key length is 0
9       - key length is more than 0
```

Listing 20: `kv_test.go` 第 8–16 行：三个方法共用两个分区

4.2.2. 维度 × 方法 适用性矩阵

维度	Get	Del	Set
key 是否在 store	适用：命中 / 未命中	适用：删到 / 没删到	**不适用**：不看该状态
key 合法性	适用：nil / 空 / 非空	适用：同上	适用：同上
value 是否相同	不适用	不适用	阶段末： `updated` 语义
history	适用	适用	适用

判断「适用」的方法很简单：**固定其他维度，只改这一个，输出会不会变？** `Set` 在「key 已存在」和「key 不存在」两种情况下，非空键都返回 `true, nil` —— 输出不变，所以这个维度对 `Set` 没有分叉作用。

4.2.3. 证据：Set 的行为不随「key 是否存在」变化

实现只检查 key 的合法性，然后无条件写入：

```
    if key == nil {
      return false, ErrKeyIsNil
    } else if len(key) == 0 {
      return false, nil
    }

    kv.mem[string(key)] = bytes.Clone(val)
    newVal, _ := kv.mem[string(key)]
    updated = bytes.Equal(val, newVal)
```

Listing 21: `Set` 的实现：没有读取「旧值是否存在」的分支

11-12 的探针已经实测过三种情况：

```
    new key:            updated=true err=<nil>
    same key same val:  updated=true err=<nil>
    same key diff val:  updated=true err=<nil>
```

Listing 22: 三种情况输出相同：`Set` 的成功路径对「是否存在」不敏感

所以把 `Set` 列进「key 是否在 store」分区，是一条**没有对应行为的分区描述**。它既不能指导用例，也不能在失败时帮你定位。

如果将来 `updated` 改成「值是否变化」，`Set` 才会按「新键 / 同键不同值 / 同键同值」分叉—— 那时的维度是 **value 相等性**，仍然不是简单的 in / not in。

4.2.4. 仍未修的互斥问题

`key is nil` 与 `key length is 0` 仍然重叠：nil 切片的长度就是 0。一个 nil key 同时落进两个子域，分区就不互斥。

修正后的三分法只有一种：

```
    partition on key validity:
      - key is nil
      - key is non-nil and len(key) == 0
      - key is non-nil and len(key) > 0
```

Listing 23: 先判 nil，再判长度：三个谓词两两不同时为真

4.2.5. 空的子域

分区列了 `nil` 与空键，但测试仍然只调用了一次 `Set`（非空键）。`Get` / `Del` 的 nil 子域也没有用例。

两种处置，二选一：

处置	动作	适用情形

现在补	加两条断言：`Set(nil)` 与 `Set([]byte{})`	这些分支已实现、spec 已写
阶段末	写 `TODO(阶段末): Set 的 nil / empty key 子域暂不覆盖`	按测试强度原则推迟

沉默不行。列了子域却不测、又不标注，读者无法判断这是遗漏还是决定。

4.2.6. 仍缺的两块

1. **history 分区**：`Set → Get`、`Set → Del → Get`、`Del → Del`；
2. **阶段末清单**：同键同值、value 空 / nil、别名、Open / Close、零值。

没有 history，第 47 行的错误就没人发现：它删了 `k1` 却查 `xxx`，而 `Set → Del → Get` 的 Get 应该查被删的键。

4.2.7. 第 47 行仍是假绿

```
1    _, ok, err = kv.Get([]byte{'x', 'x', 'x'})
2    assert.True(t, !ok && err == nil)
```

Listing 24: 删除 `k1` 之后查 `xxx`：删除逻辑失效时这条断言仍然通过

这是当前测试里唯一「测试自己写错」的地方，优先级高于任何分区润色。

4.2.8. 拼写与空行

`exisit` 出现两次；注释开头有两个空的 `*` 行。`gofmt -l` 无输出（它不检查拼写），但评审和 `go doc` 会原样看到。

4.2.9. 修正后的分区结构（骨架）

```
Get(), Del():
  partition on key in store:
    - key exists in store
    - key does not exist in store
  partition on key validity:
    - key is nil
    - key is non-nil and len(key) == 0
    - key is non-nil and len(key) > 0

Set():
  partition on key validity:
    - key is nil
    - key is non-nil and len(key) == 0
    - key is non-nil and len(key) > 0
  # 当前强度不按 value 分叉；阶段末补在下面

history (all methods):
  - Set -> Get
  - Set -> Del -> Get
  - Del -> Del

TODO(阶段末):
  - Set same key same value / different value
  - value is nil / empty
  - aliasing (input / return value)
  - Open / Close lifecycle, zero value
```

Listing 25: 按方法拆开：Get/Del 两个维度，Set 一个维度，history 单列

4.2.10. 验收清单

顺序	动作	验收
1	第 47 行 `xxx` → `k1`	注释 `delete` 后测试变红
2	把 `Set` 从「key 是否在 store」分区移出	分区表与方法行为对应
3	修 key validity 为互斥三分法	谓词两两不同时为真
4	补 history 三条	第 47 行归入 `Set → Del → Get`
5	nil / 空键子域：补断言或标 TODO	没有空子域
6	修 `exisit` 与开头空 `*`	注释干净

4.3. 陷阱与易错

⚠️ **陷阱预警 维度对方法没有分叉作用，就不该列。** `Set` 不看「key 是否在 store」，列进去只会让分区表看起来完整、实际上无法预测行为。

⚠️ **陷阱预警 分区不互斥，等于没有分区。** nil 与 len 0 重叠，失败时无法归因。

⚠️ **陷阱预警 列了子域却没有用例。** 要么补断言，要么标 `TODO(阶段末)`，不能沉默。

⚠️ **陷阱预警 假绿优先级最高。** 第 47 行删了 `k1` 却查 `xxx`，它不是「覆盖不足」，而是现有断言写错了键。

4.4. 关键点

📌 **核心要点** 这次分区的核心问题是「维度错配」：`Set` 被放进了一个它不敏感的维度。修法是按方法拆开分区，并给每个维度找一条能改变输出的证据。

维度	Get	Del	Set
key in store	要	要	不要
key validity	要	要	要
value equality	不要	不要	阶段末

4.5. 变式检验

变式 1：什么情况下 `Set` 会按「key 是否在 store」分叉？如果 `updated` 定义为「键是否新增」（新键 true、覆盖 false），那 in / not in 才成为 `Set` 的分区。当前实现和 spec 都不是这个语义。

变式 2：怎么快速判断一个维度该不该列进分区？固定其他变量，只改这个维度，观察输出。输出不变，就不该列。`Set` 在「新键 / 已存在」下都返回 `true, nil`，所以不列。

变式 3：`Get` 和 `Del` 为什么共享「key in store」分区？两者的行为都依赖键是否存在：`Get` 区分命中 / 未命中，`Del` 区分删到 / 没删到。共享分区是合理的，但仍要各自列子域。

变式 4：分区表能不能三个方法共用一张？维度适用范围不同时不能硬合并。可以共用「key validity」，但「key in store」只属于 `Get` / `Del`。

变式 5：第 47 行怎么验证真的能红？把 `Del` 里的 `delete(kv.mem, ...)` 注释掉，跑测试；必须 FAIL。恢复后再跑，必须 PASS。这一步比补任何分区都能说明测试是否有效。

4.6. 同类归类

维度类型	回答什么	本项目例子
状态	对象当前处于什么状态	key in store / not in store
输入合法性	输入是否被接受	nil / 空 / 非空
值属性	值的相等性、空值	同键同值（阶段末）
历史	之前发生了什么调用	Set → Del → Get
别名	输入输出是否共享内存	改入参 / 改返回值（阶段末）

4.7. Go 语言机制补充

方法的分区是方法特定的。 Go 没有「类型的统一分区」概念；每个方法按自己的分叉点写 strategy。`Get` / `Del` 依赖 comma-ok 与删除结果，`Set` 依赖 key 合法性，三者不能共用一张表。

`Set` 的写入不返回「是否覆盖」。 内置 map 的赋值没有返回值，Go 也不提供「这次赋值是否改变了值」的原子检查；要实现「值是否变化」，必须先读旧值。

`gofmt` 不检查注释拼写。 `exisit` 会原样保留；`gofmt -l` 无输出不代表注释正确。

6.031 对照。 本题对应 R04 测试分区：互斥、完整、非空；以及「分区要能指导用例」——对方法没有分叉作用的维度，写进去只是噪音。

4.8. 本次操作与当前状态

已做的文件操作：

1. 读取 `internal/db/kv_test.go` 当前 strategy（第 8–16 行）与 7 条断言；
2. 读取 `internal/db/kv.go` 的 `Set` 实现（第 126–137 行），确认它不读取「键是否存在」；
3. 运行 `gofmt -l internal/db/` 与 `go test ./internal/db/ -count=1`；
4. 追加本一级标题到 `note/2026-11-13.typ`；
5. 未修改 `internal/db` 下任何代码或测试文件。

验证结果：

1. `gofmt -l internal/db/` → 无输出；
2. `go test ./internal/db/ -count=1` → `ok`；
3. `Set` 实现确认：只检查 nil / 空键，随后无条件写入并返回 `true`；
4. 11-12 的探针输出确认：新键、同键同值、同键不同值都返回 `updated=true`；
5. `typst compile 2026-11-13.typ` → 待编译确认（见下）。

需要你自己动手的：

1. 第 47 行 `xxx` → `k1`，并做「注释 `delete` 看红」的验证；
2. 把 `Set` 从「key 是否在 store」分区移出；
3. 修 key validity 的互斥三分法；
4. 补 history 三条；
5. nil / 空键子域：补断言或标 `TODO(阶段末)`；
6. 修 `exisit` 与开头空 `*` 行。

Index of Listings

Listing 1 `kv_test.go` 第 6–20 行：两个分区有内容，两个是空 TODO 4
Listing 2 strategy 的骨架：分区 → 子域 → 阶段末清单；文字由你自己写 5
Listing 3 删的是 `k1`，查的是 `xxx`： 删除逻辑失效时这条断言仍然通过 6
Listing 4 拆断言： `require` 用于后续断言依赖的前置， `assert` 用于独立的属性 7
Listing 5 GOROOT `src/bytes/bytes_test.go` 的 `TestClone` （节选）：nil、空、零容量各自成组 7
Listing 6 `kv_test.go` 第 8–17 行：两个分区，一个重叠 11
Listing 7 三种输入：前两种长度都是 0，但 nil 性不同 12
Listing 8 互斥且完整的三分法：先判 nil，再判长度 12
Listing 9 最小 history 分区：正常链路、删除后不可见、重复删除 12
Listing 10 当前唯一的 Set 调用：只覆盖「非空键」子域 13
Listing 11 推迟也要留下可追溯的标记 13
Listing 12 删 `k1` 却查 `xxx`： 删除逻辑失效时这条断言仍然通过 13
Listing 13 修正后的骨架：互斥的三分法 + key 合法性 + history + 阶段末清单 14
Listing 14 GOROOT `src/sync/map.go`： 未命中用 `ok` 表达，不产生错误 18
Listing 15 GOROOT `src/net/http/request.go`： nil 的含义按角色（client / server）明确规定 19
Listing 16 GOROOT `src/context/context.go`： nil 表示「没有这个能力」 19
Listing 17 TypeScript 官方文档： `strictNullChecks` 的两种世界 19
Listing 18 官方文档的简化示例：关闭检查时， `undefined` 从类型系统里消失 19
Listing 19 nil 同时表示「值是 nil」和「键不存在」——这正是课程批评的第三层 20
Listing 20 `kv_test.go` 第 8–16 行：三个方法共用两个分区 24
Listing 21 `Set` 的实现：没有读取「旧值是否存在」的分支 25
Listing 22 三种情况输出相同： `Set` 的成功路径对「是否存在」不敏感 25
Listing 23 先判 nil，再判长度：三个谓词两两不同时为真 25
Listing 24 删除 `k1` 之后查 `xxx`： 删除逻辑失效时这条断言仍然通过 26
Listing 25 按方法拆开：Get/Del 两个维度，Set 一个维度，history 单列 27